算法计量与验证实验报告 - 第 03 章作业

姓名：[Redacted]　学号：[Redacted]

运行环境说明

- **操作系统**：macOS
- **python 版本**：3.12.7
- **代码仓库**： `https://github.com/maidou07/dsa2026.git`

实验一：中缀表达式直接计算

程序原理

双栈法（操作数栈与运算符栈）从左至右扫描表达式。遇到操作数则压入操作数栈；遇到运算符则比较优先级，若栈顶运算符优先级不低于当前运算符，则弹出栈顶运算符并执行计算（从操作数栈弹出两个操作数），直至栈顶优先级更低或栈为空，再将当前运算符压入运算符栈。遇左括号压栈，遇右括号则弹栈计算直至匹配左括号。最终操作数栈顶即为结果。

实际实现封装了两个函数：`cal(op)` 和 `tokenize(s)`。前者执行每一次计算操作，封装提升代码简洁度；后者用于将不带空格的常规输入表达式拆分成计算函数可以处理的 `tokens`。

测试用例与运行结果

编号	测试表达式	计算结果
用例 1	`2+3*4-4/2`	12
用例 2	`(2+3)*(4-1)`	15
用例 3	`((1+2)*(3+4))/3-8/4`	5
用例 4	`1+2*3-4/2+6*(7-8/2)+9/3*2`	29

表 1: 中缀表达式计算测试用例

代码实现

```python
pri = {"+":1, "-":1, "*":2, "/":2}
ops = []    # 直接用list实现栈
nums = []
# 封装计算函数，提升代码整洁度
def cal(op):
    global nums
    a, b = nums.pop(), nums.pop()
    if op == '+': nums.append(b + a)
    elif op == '-': nums.append(b - a)
    elif op == '*': nums.append(b * a)
    elif op == '/': nums.append(b // a)
# 格式化表达式
def tokenize(s):
    cache = []  # 数位 缓存
    tokens = []
    for char in s:
        if char.isspace():
            continue
        if char in ['(',')','+','-','*','/']:
            if cache:
                tokens.append(''.join(cache))
                cache.clear()
            tokens.append(char)
        else:
            cache.append(char)
    if cache:
        tokens.append(''.join(cache))
        cache.clear
    return tokens
# 中缀表达式直接计算
def infix_to_value(tokens):
    global nums, ops
    for token in tokens:    # 迭代处理中缀
        if token == '(':
            ops.append(token)
        elif token == ')':
            while ops[-1] != '(':   # 清算括号内表达式
                op = ops.pop()
                cal(op)
            ops.pop()
        elif token in pri:
            while ops and ops[-1] in pri and pri[ops[-1]] >= pri[token]:    # 优先级高和平级先到的全部出栈
                op = ops.pop()
                cal(op)
            ops.append(token)   # 新op入栈
        else:   # 数字直接入栈
            nums.append(int(token))
    while ops:
        op = ops.pop()
        cal(op)
    return nums[-1]
print(infix_to_value(tokenize(input())))
```

实验二：队列性能对比实验

程序原理

List 队列基于 Python 动态数组实现，底层为连续内存空间；入队时 `append` 在尾端追加元素，出队时 `pop(0)` 移除首元素。链表队列由游离节点类 `ListNode` 构成，通过头尾指针分别管理队首与队尾；入队创建新节点并链接至尾节点，出队只需移动头指针指向下一节点。

测试用例和运行结果

分别测试三个数量级的队列，完整执行一套入队流程和全部出队流程，分别计时，得到以下的测试结果：

数据规模	队列类型	入队耗时 (ms)	出队耗时 (ms)
1000	LIST	0.031 87	0.120 17
	LINKEDLIST	0.170 50	0.118 79
10000	LIST	0.288 38	4.902 00
	LINKEDLIST	1.592 13	1.060 58
100000	LIST	2.619 71	575.116 42
	LINKEDLIST	14.842 79	10.072 96

表 2: 队列性能对比测试结果

实验分析

入队分析：可以看到，在数据规模增长时，两者入队用时都成**线性增长**。这是因为两者的入队均为 $O(1)$ 时间复杂度。List 基于 C 数组实现，`append` 操作在尾部追加，通过内置的均摊，可以保证每次插入都是常数开销。链表队列需每次创建一个新的 `ListNode` 对象，涉及内存分配与初始化，开销高于原生的 C 数组操作。因此 **List 入队略快于链表队列**。

出队分析：性能差异显著，完全出队的时间复杂度为 $O(N^2)$ 与 $O(1)$ 的差异。List 的 `pop(0)` 移除首元素后，必须将后续 $N-1$ 个元素整体前移一位以维持连续内存布局，时间复杂度 $O(N)$。当连续执行 N 次出队时，总时间复杂度退化为 $O(N^2)$，数据规模从 10000 增至 100000 时出队耗时从4.9 ms 飙升至575.1 ms。链表队列出队仅修改头指针，不涉及数据移动，保持稳定的 $O(1)$ 性能，出队耗时随规模**线性增长**（100× 数据仅约 100× 耗时）。

代码实现

```python
import timeit

# 基于List的队列（队尾入队O(1)，队头出队O(n)）
class ListQueue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Queue is empty")
    def is_empty(self):
        return len(self.items) == 0
# 队列单向链表实现：同时记录首尾，尾部入队O(1)，头部出队O(1)
class ListNode:
    def __init__(self, value = None):
        self.val = value
        self.next = None
class LinkedListQueue():
    def __init__(self):
        self.head = ListNode()
        self.tail = self.head
        self.size = 0
    def _add_to_tail(self, node: ListNode):
        self.tail.next = node
        self.tail = node
        self.size += 1
    def _remove_from_head(self):
        if self.head.next is None:
            raise IndexError("Queue is empty")
        target = self.head.next
        self.head.next = target.next
        if self.size == 1:
            self.tail = self.head
        self.size -= 1
    def enqueue(self, val: int):
        new_node = ListNode(value = val)
        self._add_to_tail(new_node)
    def peek(self):
        if self.size == 0:
            raise IndexError("Queue is empty")
        return self.head.next.val
    def dequeue(self) -> int:
        if self.size == 0:
            raise IndexError("Queue is empty")
        val = self.head.next.val
        self._remove_from_head()
        return val
```

```python
# 测试给定队列的出入队性能：
def test_performance():
    # 枚举数据大小
    test_sizes = [1000, 10000, 100000]
    for size in test_sizes:
        # 测量 N 次入列的用时：
        # A. 基于list的链表
        setup_list = "from __main__ import ListQueue; q = ListQueue()"
        run_list = "q.enqueue(1)"
        t_enq_list = timeit.timeit(run_list, setup_list, number = size)
        # B. 基于linkedlist的链表
        setup_ll = "from __main__ import LinkedListQueue; q = LinkedListQueue()"
        run_ll = "q.enqueue(1)"
        t_enq_ll = timeit.timeit(run_ll, setup_ll, number = size)

        # 测量 N 次出列的用时：
        # A. 基于list的链表
        setup_list = f"from __main__ import ListQueue; q = ListQueue(); [q.enqueue(i) for i in range({size})]"
        run_list = "q.dequeue()"
        t_deq_list = timeit.timeit(run_list, setup_list, number = size)
        # B. 基于linkedlist的链表
        setup_ll = f"from __main__ import LinkedListQueue; q = LinkedListQueue(); [q.enqueue(i) for i in range({size})]"
        run_ll = "q.dequeue()"
        t_deq_ll = timeit.timeit(run_ll, setup_ll, number = size)
        print(f"# SIZE: {size} ")
        print(f"       LIST  |{1000*t_enq_list:^15.5f}ms|{1000*t_deq_list:^15.5f}ms")
        print(f" LINKEDLIST  |{1000*t_enq_ll:^15.5f}ms|{1000*t_deq_ll:^15.5f}ms")
test_performance()
```